• Store the training examples and their known labels.

• For a new user, calculate the distance from that user to each training example.

• Sort by distance and choose the K closest examples.

• For classification, count the labels of those neighbours.

• Predict the majority class.

Example

Suppose K=5 and the five nearest users have purchase labels [1, 1, 0, 1, 0]. Three neighbours have label 1 and two have label 0. The prediction is therefore **Purchased = 1**.

Meaning of K

K is the number of neighbours used for voting. This notebook uses **K=5**. A very small K is sensitive to noise and unusual examples. A very large K may smooth away useful local patterns and underfit. K is usually chosen by validation or cross-validation.

Distance formula

The default Minkowski distance with p=2 is Euclidean distance. For two features: distance = √[(Age■ – Age■)² + (Salary■ – Salary■)²]. KNN chooses points with the smallest distance.

Why scaling is necessary

Age may be a two-digit number, while salary may be tens of thousands. Without scaling, salary differences would dominate the distance because of their larger numeric range. StandardScaler converts values to z-scores: z = (x – mean) / standard deviation. It learns the statistics from training data and applies them to test data.

KNN is often called a lazy learner because it stores examples and performs much of its work when a prediction is requested, rather than learning a compact equation during training.

3. Code walkthrough — imports and data

Cell 1: import libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score
```

pandas (pd): reads and manipulates tabular data. **NumPy (np):** numerical computing library; it is imported but not used in this notebook. **train_test_split:** separates training and test data. **make_pipeline:** chains preprocessing and a model. **KNeighborsClassifier:** KNN classification implementation. **StandardScaler:** standardises features. The metric functions evaluate predictions.

Cell 2: load the file

```
df = pd.read_csv('Social_Network_Ads.csv')
df.head()
```

pd.read_csv() reads the CSV file and returns a pandas DataFrame. **df** is the complete table. **head()** shows the first five rows by default; it is used to inspect the data, not to train the model.

Cell 3: select inputs and target

```
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']
```

Double brackets select multiple columns and return a DataFrame (two-dimensional). Single brackets select one column and return a Series (one-dimensional). X contains the two inputs; y contains the correct labels.

Cell 4: preview

```
X.head()
y.head()
```

In a notebook, often only the last expression in a cell is displayed automatically. That is why the visible output may show y.head() but not X.head(). Use print(X.head()) and print(y.head()) to show both.

4. Train/test split, pipeline and prediction

Cell 5: split the data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=42, stratify=y, test_size=0.2
)
```

test_size=0.2: 20% test data and 80% training data. For 400 rows this means 320 train and 80 test rows. **random_state=42:** makes the random split reproducible; 42 has no special mathematical meaning. **stratify=y:** keeps class proportions approximately similar in both subsets.

Why split?

If we test on examples used for training, performance can look too good. A separate test set estimates performance on unseen examples. Do not repeatedly tune the model against the test set.

Cell 6: build and train model

```
model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

make_pipeline() combines two steps in order: first StandardScaler, then KNeighborsClassifier. **fit()** fits the scaler on X_train, scales the training data and fits KNN using y_train. For KNN, fitting mainly stores the scaled training points and labels. **predict()** scales X_test using the already-fitted scaler and predicts 0 or 1 for each row.

Why pipeline?

• Keeps scaling and model together.

• Applies exactly the same fitted scaling to training and test data.

• Reduces accidental preprocessing mistakes.

• Helps prevent data leakage: the scaler learns only from training data.

Data leakage means information from test data (or future information) has influenced model training. Scaling the full dataset before splitting can leak test-set statistics into the training process.

5. Evaluation metrics and actual result

Cell 7: evaluate predictions

```
accuracy = accuracy_score(y_test, pred)
precision = precision_score(y_test, pred)
recall = recall_score(y_test, pred)
cm = confusion_matrix(y_test, pred)
```

The original notebook stores the result in a variable also named **confusion_matrix**. This works for that line, but it shadows the imported function name afterward. A clearer variable name is **cm**.

Confusion matrix

	Predicted 0	Predicted 1
Actual 0	47 — TN	4 — FP
Actual 1	4 — FN	25 — TP

TN=47: actual non-buyers correctly predicted as non-buyers. **FP=4:** actual non-buyers incorrectly predicted as buyers. **FN=4:** actual buyers missed by the model. **TP=25:** actual buyers correctly predicted as buyers.

Notebook output

Accuracy: 0.90 (90%)
Precision: 0.8620689655 (86.21%)
Recall: 0.8620689655 (86.21%)
Confusion matrix: [[47, 4], [4, 25]]

Formulas

Accuracy = (TP+TN)/total = (25+47)/80 = 0.90.
Precision = TP/(TP+FP) = 25/29 ≈ 0.8621. Of predicted buyers, 86.21% really bought.
Recall = TP/(TP+FN) = 25/29 ≈ 0.8621. Of actual buyers, 86.21% were identified.
Error rate = (FP+FN)/total = 8/80 = 0.10 = 1–accuracy.

Interpretation: 72 of 80 test examples were correctly classified. The model made four false-positive and four false-negative errors.

6. Functions and parameters — viva questions

Q. What is a DataFrame?

A. A two-dimensional labelled table with rows and columns.

Q. What does pd.read_csv() return?

A. A pandas DataFrame containing the CSV data.

Q. What does head() do?

A. Shows the first five rows by default; head(10) shows ten.

Q. df['Age'] versus df[['Age']]?

A. The first returns a Series; the second returns a one-column DataFrame.

Q. Why X uppercase and y lowercase?

A. A common convention: X is the feature matrix and y is the target vector.

Q. What does train_test_split return?

A. Here, X_train, X_test, y_train and y_test.

Q. What is random_state?

A. A seed that makes the random split reproducible.

Q. Why stratify=y?

A. To preserve approximately similar class proportions in train and test sets.

Q. What does fit() do?

A. Fits preprocessing and model steps using the training data.

Q. What does predict() return?

A. Predicted class labels for each supplied test row.

Q. What does StandardScaler do?

A. Standardises each feature using training mean and standard deviation.

Q. What does n_neighbors=5 mean?

A. The five nearest training points vote on the predicted class.

Q. What does make_pipeline() do?

A. Creates a sequence of estimators and applies them in order.

Q. What does accuracy_score() calculate?

A. The fraction of predictions that match the true labels.

Q. What does confusion_matrix() return?

A. Counts of correct and incorrect predictions by actual and predicted class.

Q. What is the default positive class for precision_score?

A. For binary classification, pos_label=1 by default.

Q. Why is NumPy imported?

A. It is a common numerical library, but it is unused in this particular notebook and can be removed.

7. Core theory — examiner cross-questions

Q. Is KNN supervised or unsupervised?

A. Supervised, because the training examples have known labels.

Q. Classification or regression here?

A. Classification: Purchased is a category, 0 or 1.

Q. Can KNN perform regression?

A. Yes. KNN regression predicts a number, often by averaging nearby target values.

Q. Why is KNN called lazy learning?

A. It stores examples and does much of its work at prediction time.

Q. What if K=1?

A. The nearest single point decides; this can be sensitive to noise.

Q. What if K is very large?

A. The model may become too smooth and miss local patterns (underfit).

Q. Should K be odd?

A. An odd K can reduce ties in binary classification, but does not guarantee that ties are impossible in all cases.

Q. What is Euclidean distance?

A. Straight-line distance, calculated using the square root of summed squared feature differences.

Q. Why scale features?

A. Distance-based algorithms can be dominated by features with large numeric ranges.

Q. What is overfitting?

A. Learning noise or training-specific details that do not generalise to new examples.

Q. What is underfitting?

A. A model that is too simple to capture the important pattern.

Q. What is a decision boundary?

A. The border between regions predicted as different classes.

Q. What is data leakage?

A. Using information in training that would not legitimately be available, such as test-set statistics.

Q. Why keep a test set?

A. To estimate performance on unseen examples.

Q. What is class imbalance?

A. One target class has many more examples than another.

Q. When is precision important?

A. When false positives are costly.

Q. When is recall important?

A. When false negatives are costly and we want to find most actual positives.

Q. What is F1-score?

A. The harmonic mean of precision and recall: 2PR/(P+R).

Q. Does 90% accuracy mean 90% for each class?

A. No. It is an overall score; use class-wise metrics and the confusion matrix for detail.

Q. Is KNN always fast?

A. Training is simple, but prediction can be costly because distances to stored examples must be calculated.

8. Code cross-questions and possible improvements

Q. Why are User ID and Gender not used?

A. The notebook selects Age and EstimatedSalary only. ID is not a meaningful measurement; Gender was simply not part of this experiment.

Q. Can Gender be included?

A. Yes, after suitable categorical encoding such as one-hot encoding, while considering relevance and fairness.

Q. What if the CSV file is not found?

A. Check filename, extension, working directory and file path.

Q. What if data contains missing values?

A. Inspect df.isna().sum(); then justify row removal or imputation. Fit imputation using training data only.

Q. Why not scale before splitting?

A. That lets test data influence scaling statistics, causing leakage.

Q. Can KNN handle more than two classes?

A. Yes. KNeighborsClassifier supports multiclass classification.

Q. How to choose a better K?

A. Try multiple K values with cross-validation on training data, then evaluate once on the test set.

Q. What does weights='distance' do?

A. Closer neighbours get more voting influence than farther neighbours.

Q. Can the distance metric be changed?

A. Yes. Common choices include Euclidean and Manhattan distance.

Q. What is the basic prediction cost?

A. A query may require calculating distances to many training points; cost grows with dataset size and feature count.

Clean version of the notebook code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             precision_score, recall_score)

df = pd.read_csv("Social_Network_Ads.csv")
X = df[["Age", "EstimatedSalary"]]
y = df["Purchased"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Error rate:", 1 - accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred))
print("Recall:", recall_score(y_test, pred))
print("Confusion matrix:\\n", confusion_matrix(y_test, pred))
```

This version removes the unused NumPy import, uses cm-style naming rather than overwriting a function, and prints the error rate requested in the assignment.

9. Last-minute revision sheet

Explain the experiment in 30–45 seconds

“My experiment uses K-Nearest Neighbours to predict whether a user purchases a product based on age and estimated salary. I load the Social Network Ads CSV using pandas, select Age and EstimatedSalary as features and Purchased as the target. I split the data into 80% training and 20% testing with stratification. A pipeline first standardises the features and then applies KNN with five neighbours. I train using fit() and predict test labels using predict(). The notebook reports 90% accuracy, approximately 86.21% precision and 86.21% recall. Its confusion matrix is [[47, 4], [4, 25]], so 72 out of 80 test examples were classified correctly.”

Numbers to remember

Item	Notebook value
Train / test	80% / 20%
random_state	42
stratify	y
K	5
Accuracy	90%
Error rate	10% (derived)
Precision	86.21%
Recall	86.21%
TN / FP / FN / TP	47 / 4 / 4 / 25
Test rows	80

Common mistakes to avoid

• Do not say KNN learns a straight-line equation; it predicts from nearby stored examples.

• Purchased is the target y, not an input feature.

• Scaling matters because KNN uses distance.

• Precision uses predicted positives; recall uses actual positives.

• 47 is the number of true negatives, not true buyers.

• Do not claim the model is universally reliable based on one test split.

Strong closing answer

“The result is promising for this particular test split. I would next tune K using cross-validation, compare distance metrics and check class-wise performance before using the model in a real advertising system.”

Source note: the code structure and reported output values are taken from the uploaded notebook. Explanations of KNN, scaling, evaluation and suggested improvements are standard supporting theory for viva preparation.